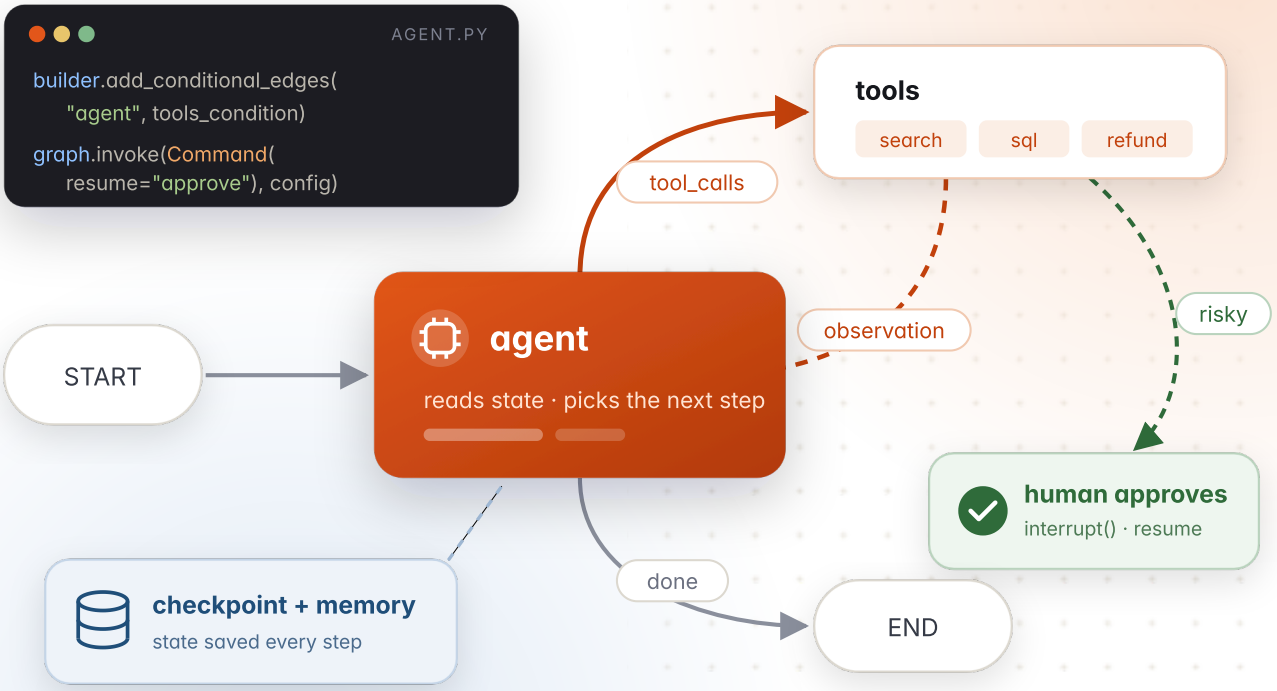
A FIELD GUIDE FOR BUILDERS

OCTOBER 2026 EDITION

Agentic AI

from zero to production →

What it actually takes to build AI agents that hold up in the real world: the concepts, the trade-offs, the failure modes, and working code you can ship.



01 Agents vs workflows
02 Tool calling
03 Memory
04 Planning
05 Human-in-the-loop
06 LangGraph
07 Multi-agent systems
08 Production

8 chapters, concept to deployment

7 code samples tested on LangGraph 1.2

20+ diagrams and decision tables

15 pages, no filler

Contents

01	**Agents vs workflows** Who decides the next step, and why that choice drives cost and risk	3
02	**Tool calling** What actually travels over the wire, and how to design tools models use well	5
03	**Memory** Context window, thread state and long-term store	7
04	**Planning** ReAct, plan-and-execute, reflection, and effort scaling	8
05	**Human-in-the-loop** Where to put approval gates, and how pause/resume really works	9
06	**LangGraph** State, reducers, nodes, edges, checkpoints, a complete agent	10
07	**Multi-agent systems** Patterns, the evidence for and against, a supervisor in code	12
08	**Shipping to production** Limits, evals, tracing, security, failure modes, reference architecture	14

HOW TO READ THIS

Chapters 1–5 are concepts and apply to any framework. Chapters 6–7 show them in LangGraph. Chapter 8 is a checklist you can use on a real project.

Code targets `langgraph 1.2` and `langchain 1.4` (Python). Each sample was run before printing.

The whole idea on one page

An agent is a language model running in a loop. On each turn it reads the current state, either answers or asks for a tool to be run, and your code feeds the result back. Everything else in this guide is about controlling that loop: what it can touch, what it remembers, when it stops, and when a person must step in.

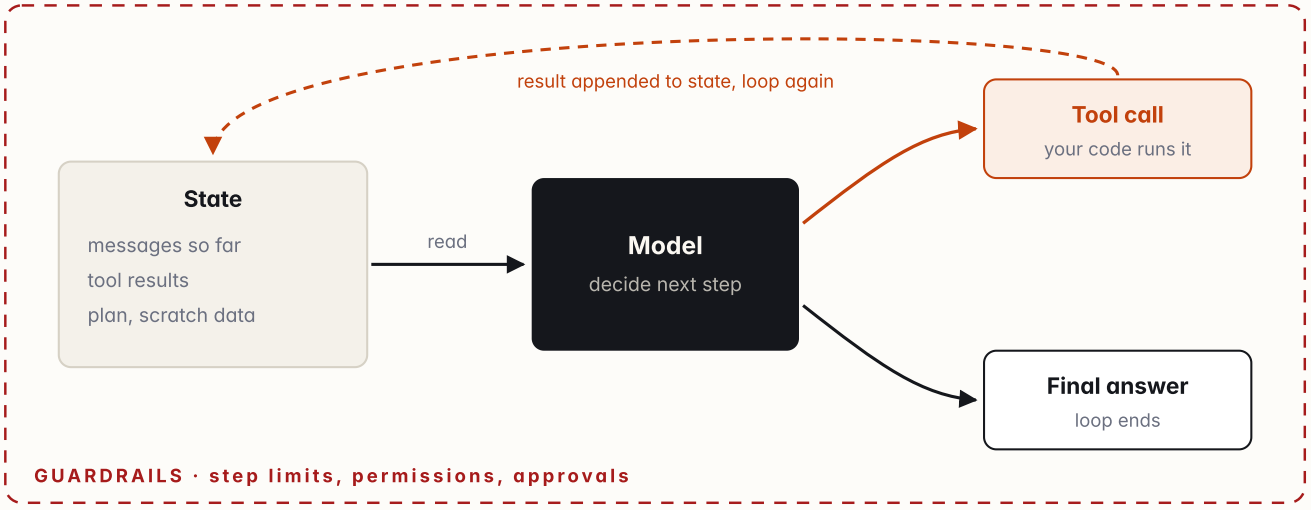


FIG 0.1 Every agent framework, including LangGraph, is an implementation of this loop.

MODEL
Chooses the next action. Better models make fewer, better tool calls.

TOOLS
Functions your code exposes. They define what the agent can actually affect.

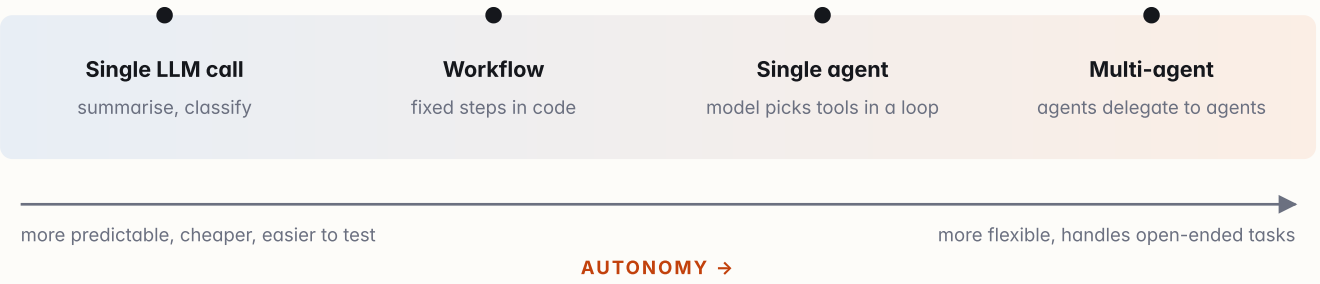
LOOP + STATE
Your code. It decides what is remembered, persisted and when to stop.

01 Agents vs workflows

Both use LLMs and tools. The difference is who decides what happens next: your code, or the model.

> "Workflows are systems where LLMs and tools are orchestrated through predefined code paths. Agents are systems where LLMs dynamically direct their own processes and tool usage."
>
> Anthropic, *Building Effective Agents* (2024)

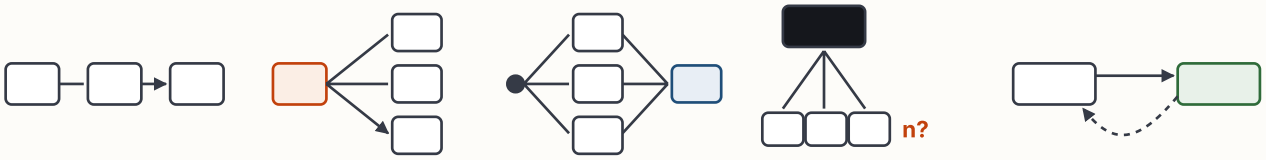
A spectrum, not a binary



Move right only when the task requires it. Each step adds flexibility and also adds cost, latency and new ways to fail.

The five workflow patterns worth knowing

Most production "agents" are really one of these, and that is a good thing. They are easy to test and their cost is predictable.



Prompt chaining
Fixed steps; output of one feeds the next, with code checks between.
e.g. outline → draft → translate

Routing
Classify input, send to a specialised prompt or cheaper model.
e.g. support ticket triage

Parallelisation
Run independent calls at once (sections), or the same call n times (vote).
e.g. guardrail + answer

Orchestrator–workers
An LLM splits the task; number of subtasks is not known in advance.
e.g. multi-file code edit

Evaluator–optimiser
One LLM drafts, another grades against criteria, loop until it passes.
e.g. literary translation

FIG 1.1 Pattern names follow Anthropic's taxonomy. Orchestrator–workers is the bridge to agents: the model decides the subtasks.

Choosing: a practical test

Ask: *can I draw the flowchart before the input arrives?* If yes, build a workflow. If the number or order of steps depends on what the model discovers along the way, you need an agent.

	WORKFLOW	AGENT
Control flow	Written in code; the same for every input	Chosen by the model at run time
Best for	Well-understood, repeatable tasks: extraction, triage, report generation	Open-ended tasks: research, debugging, multi-step support cases
Cost & latency	Known in advance, easy to budget	Varies widely per request; needs hard caps
Testing	Unit-test each step	Need evals over whole trajectories
Typical failure	Breaks on inputs the flowchart did not foresee	Loops, wanders, or takes an unwanted action

What autonomy costs

~4×
tokens used by a single agent compared with a chat interaction

~15×
tokens used by a multi-agent system compared with chat

Source: Anthropic engineering, on its multi-agent research system (2025).

The same report found token usage alone explained 80% of the variance in performance on a browsing benchmark. Agents work partly *because* they spend more, so make sure the task is worth it.

A COMMON MIDDLE PATH

Wrap a small agent inside a workflow. For example, a fixed pipeline **classify → agent → validate → send**, where only the middle step is open-ended. You get flexibility where it is needed and guarantees everywhere else.



Signs you picked the wrong one

YOU BUILT AN AGENT BUT NEEDED A WORKFLOW

- The agent calls the same tools in the same order almost every time
- You keep adding "always do X first" to the prompt
- Stakeholders ask why two identical requests cost very different amounts

YOU BUILT A WORKFLOW BUT NEEDED AN AGENT

- Your router has dozens of branches and an ever-growing "other" bucket
- Steps need to repeat an unknown number of times
- Failures cluster on inputs nobody anticipated

RULE OF THUMB

Start with the simplest thing: a single well-prompted call with retrieval and examples. Add steps only when you can name the failure they fix, and confirm it with an eval.

02 Tool calling

The model never runs your tools. It returns a structured request, your code executes it, and the result goes back as a new message.

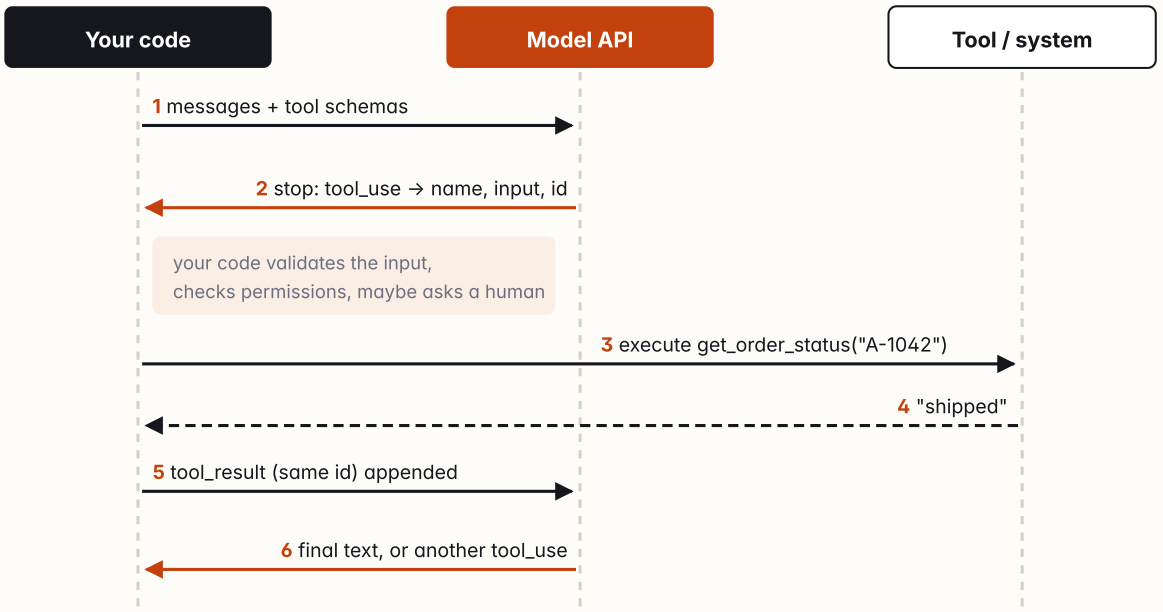


FIG 2.1 One round trip. Steps 2–5 repeat until the model stops asking for tools. That repetition is the agent loop.

What goes over the wire

Shown in Anthropic's Messages API format. OpenAI and Gemini use different field names (OpenAI sends arguments as a JSON *string*), but the shape is the same. LangChain normalises all of them into `AIMessage.tool_calls`.

REQUEST · TOOL DEFINITION

```
"tools": [{
  "name": "get_order_status",
  "description": "Return the shipping status for an
order ID such as 'A-1042'. Use before answering any
delivery question.",
  "input_schema": {
    "type": "object",
    "properties": { "order_id": { "type": "string" }
},
    "required": ["order_id"]
  }
}]
```

RESPONSE

```
{ "stop_reason": "tool_use",
  "content": [{ "type": "tool_use", "id":
"toolu_01",
                "name": "get_order_status",
                "input": { "order_id": "A-
1042" } }] }
```

NEXT REQUEST

```
{ "role": "user",
  "content": [{ "type": "tool_result",
                "tool_use_id": "toolu_01",
                "content": "shipped" }] }
```

The **description** is the most important field. The model has no other way to know when a tool applies. Treat it as a prompt for the model, not documentation for humans.

Defining a tool in code

The `@tool` decorator builds the JSON schema from the type hints and uses the docstring as the description.

TOOLS.PY

```python
from langchain_core.tools import tool

@tool
def get_order_status(order_id: str) -> str:
    """Return the shipping status for an order ID such as 'A-1042'.
    Use this before answering any question about delivery."""
    status = ORDERS.get(order_id)
    if status is None:
        # An error the model can act on, not a stack trace
        return f"No order found with id {order_id}. Ask the user to check it."
    return status

get_order_status.args_schema.model_json_schema()   # what the model sees
```

Designing tools agents use well

PRINCIPLE	IN PRACTICE
One clear job per tool	Two tools with overlapping descriptions cause wrong picks. Merge them, or say when to use each.
Match the task, not your API	Prefer `schedule_meeting(people, window)` over making the model chain `list_users`, `get_calendar` and `create_event`.
Return what the model needs	Trim large payloads. Return names, not internal UUIDs. Paginate and say so.
Errors the model can act on	"Date must be YYYY-MM-DD, got 12/03" lets it retry. A stack trace makes it guess.
Strict input types	Use enums and required fields. Validate before executing, never after.
Make writes idempotent	Agents retry. Accept an idempotency key so a retried payment is not charged twice.

EVIDENCE

40%

less task time after Anthropic had an agent test its tools and rewrite their descriptions. Agents using the new descriptions avoided earlier mistakes.

PARALLEL CALLS

Modern models can request several tools in one turn. Run them concurrently. This cut research time by up to 90% in the same system.

How many tools?

Accuracy drops as the toolset grows and descriptions start to overlap. A practical ceiling is a few dozen well-separated tools per agent. Beyond that, group tools behind specialist sub-agents (Chapter 7), or load tool definitions on demand.

MCP IN ONE PARAGRAPH

The **Model Context Protocol** is an open standard for exposing tools, data and prompts to any compatible agent. Write a GitHub or database integration once as an MCP server, and every MCP client can use it. It standardises the plumbing. Tool design still matters just as much.

03 Memory

Models are stateless. “Memory” is what your system chooses to put back into the context window.

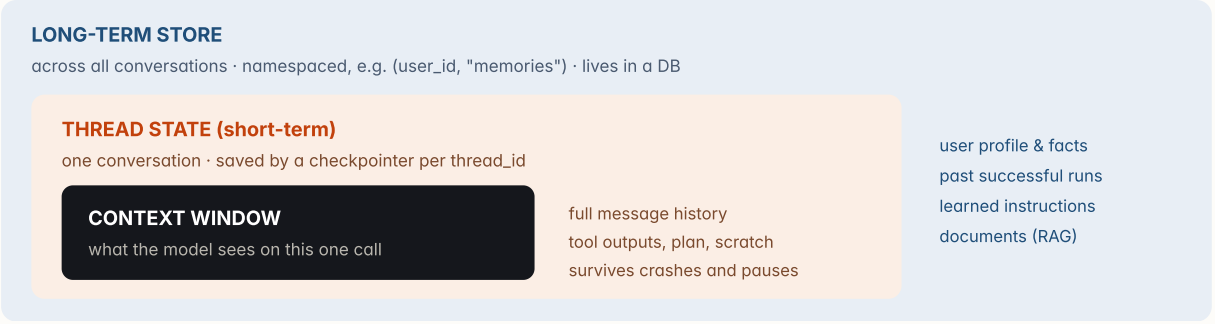


FIG 3.1 Three layers. The skill is moving the right things between them at the right time.

Three kinds of long-term memory

TYPE	HOLDS	EXAMPLE
Semantic	Facts about the user or world	“Vegetarian, allergic to peanuts”
Episodic	Past experiences, often reused as few-shot examples	How the last refund case was resolved
Procedural	How to behave: rules, refined prompts	“Always confirm the delivery address”

Categories from the CoALA paper (Sumers et al., 2023), as used in LangGraph’s docs.

When to write memories

In the hot path: the agent calls a `save_memory` tool mid-conversation. It's immediate, but adds latency and the agent may over-save.

In the background: a separate job extracts memories after the conversation. Nothing is added to latency, but it lags behind.

Managing the context window

Bigger windows don’t remove the problem. Accuracy degrades as context fills with stale tool output, well before the hard limit.

- **Trim** old turns, but keep the system prompt and latest task
- **Summarise** older history into a running note
- **Offload** large tool results to a file or store, and pass a reference
- **Persist the plan** in state so it survives truncation
- **Isolate** noisy sub-tasks in a sub-agent that returns only a summary

MEMORY.PY · CROSS-THREAD RECALL

```python
from langgraph.runtime import Runtime
from langgraph.store.memory import InMemoryStore

@dataclass
class Context: user_id: str

def recall(state: State, runtime: Runtime[Context]):
    ns = (runtime.context.user_id, "memories")       # namespace = tuple
    facts = [m.value["fact"] for m in runtime.store.search(ns, limit=5)]
    return {"messages": [("system", "Known about user: " + "; ".join(facts))]}

graph = builder.compile(checkpointer=checkpointer, store=InMemoryStore())
graph.invoke(inputs, {"configurable": {"thread_id": "t2"}},
             context=Context(user_id="u42"))
```

04 Planning

How an agent decides the sequence of steps: one at a time, all up front, or draft-and-revise.

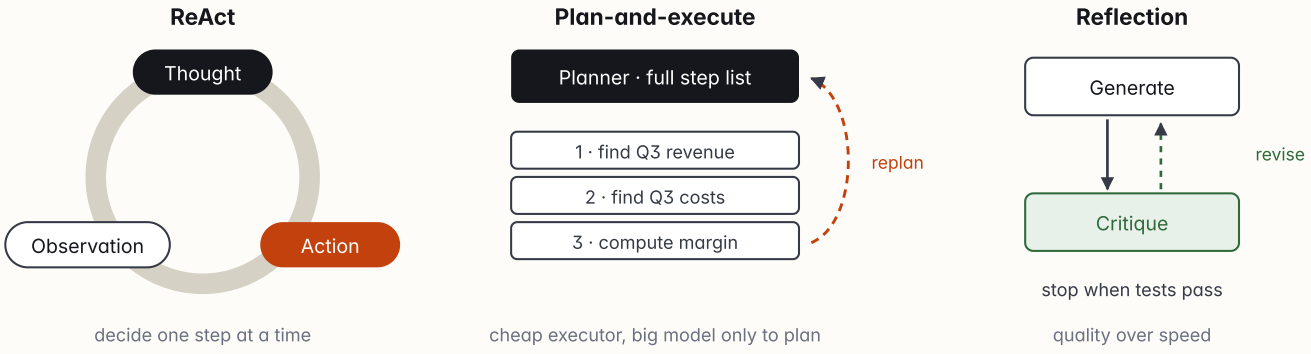


FIG 4.1 ReAct (Yao et al., 2022), plan-and-execute (Wang et al., 2023), and reflection (Reflexion, Shinn et al., 2023).

PATTERN	STRENGTH	WEAKNESS	USE WHEN
ReAct	Adapts to each new observation	Short-sighted; can wander or loop on long tasks	Default for tool-using agents with < ~20 steps
Plan-and-execute	Visible, reviewable plan; fewer big-model calls	Plan goes stale when early steps surprise	Long tasks; when a human should approve the plan
Reflection	Catches its own errors before the user does	2–3× cost; a weak critic just rubber-stamps	You have an objective check: tests, schema, rubric

Planning with reasoning models

Models with extended thinking already plan internally before acting, so a separate planner node often adds little for short tasks. An **explicit, persisted plan** still pays off when:

- The task outlives the context window. Anthropic's research lead agent saves its plan to memory because context past 200k tokens is truncated.
- A human needs to approve the approach before money is spent
- You want to audit why the agent did what it did

SCALE EFFORT TO THE TASK

Agents are bad at judging how much work a task needs. Early versions of Anthropic's system spawned **50 sub-agents** for simple queries. The fix was explicit rules in the prompt:

Simple fact	1 agent, 3–10 tool calls
Comparison	2–4 sub-agents, 10–15 calls each
Complex research	10+ sub-agents, divided scope

05 Human-in-the-loop

Not a chat window with a person watching. A durable pause at specific points, where the run stops, saves its state, and waits as long as it takes.

Where to put a gate

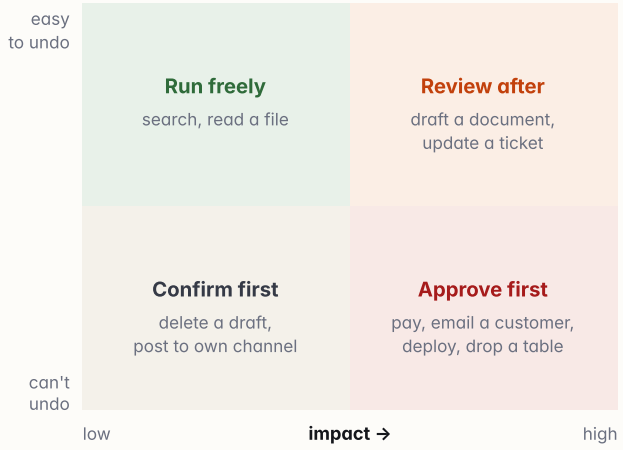
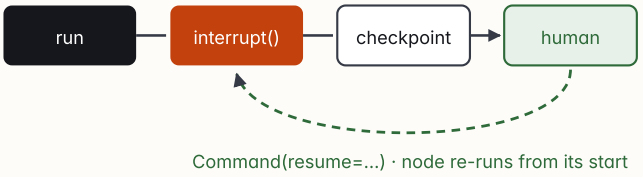


FIG 5.1 Gate by reversibility and impact, not by how "AI-ish" the step feels.

Four kinds of interrupt

`approve`	Yes or no on a proposed action, such as a refund or a send
`edit`	Human corrects the arguments before execution (amount, recipient)
`review`	Human edits the output, such as a draft, before it moves on
`clarify`	Agent asks a question it can't resolve on its own

How pause/resume works



Command(resume=...) · node re-runs from its start

HITL.PY · APPROVAL INSIDE A TOOL

```python
from langgraph.types import interrupt, Command

@tool
def refund_order(order_id: str, amount: float) -> str:
    """Refund an order. A human must approve every refund."""
    decision = interrupt({"action": "refund", "order_id": order_id,
                          "amount": amount})
    if decision != "approve":
        return "Refund rejected by reviewer."
    payments.refund(order_id, amount)      # side effect AFTER the interrupt
    return f"Refunded {amount} on {order_id}."

config = {"configurable": {"thread_id": "ticket-881"}}
out = graph.invoke({"messages": [("user", "Refund A-1042")]}, config)
out["__interrupt__"][0].value   # {'action': 'refund', 'order_id': ...}

# ...minutes or days later, from your approval UI:
graph.invoke(Command(resume="approve"), config)
```

FOUR RULES THAT BITE PEOPLE

- **The node restarts on resume.** Code before `interrupt()` runs twice, so keep side effects after it or make them idempotent.
- **Resume with the same `thread_id`.** A new id starts an empty thread.
- **No bare `try/except` around it.** The pause is implemented as an exception, and a broad `except` swallows it.
- **Use a durable checkpointer.** `InMemorySaver` loses every pending approval on restart.

06 LangGraph

A low-level runtime that models an agent as a graph over shared state, so the loop from Chapter 1 becomes something you can pause, resume, stream and inspect.

The vocabulary

`State`	A typed dict shared by every node. The single source of truth for a run.
`Reducer`	How a node's update merges into state. `add_messages` appends; the default overwrites.
`Node`	A function `state → partial update`. It can call a model, a tool, or a whole sub-graph.
`Edge`	Fixed next step. A **conditional edge** picks the next node from the state.
`Command`	Returned by a node to update state *and* choose where to go next, in one step.
`Checkpointer`	Saves state after every step, keyed by `thread_id`. This enables memory, pause/resume and recovery.
`Store`	Key-value memory shared across threads (Chapter 3).
`interrupt()`	Pauses the graph and hands a payload to your app (Chapter 5).

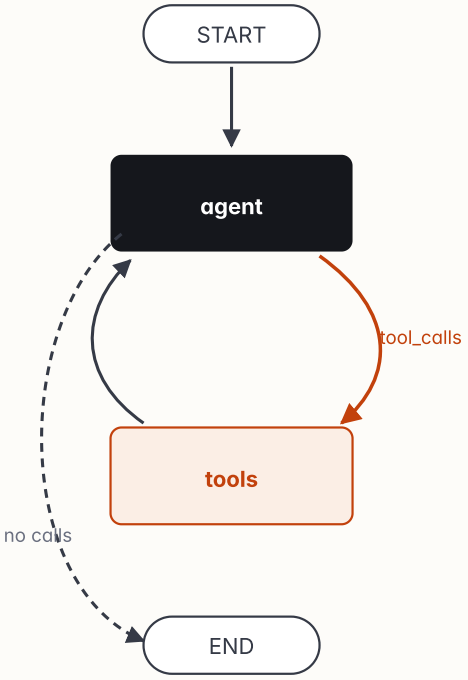


FIG 6.1 The graph built on the next page

Why a graph and not a `while` loop?

A `while` loop around `llm.invoke` is fine for a demo. The graph gives you what that loop lacks in production:

Durability
A crash or deploy mid-run resumes from the last checkpoint, not from scratch.

Pauses
Wait days for an approval without holding a process open.

Time travel
`get_state_history()` lets you inspect any step, edit state and replay from there.

Streaming
Stream full state, per-node updates or model tokens to your UI with `stream_mode`.

Explicit control
Mix deterministic steps and agent loops in one graph (the "middle path" from page 4).

Parallelism
Fan out to several nodes at once; reducers merge the results.

A complete tool-using agent

Uses the tools from Chapters 2 and 5. It persists to Postgres, caps its steps, and streams updates.

AGENT.PY

```python
from typing import Annotated, TypedDict
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode, tools_condition
from langgraph.checkpoint.postgres import PostgresSaver

class State(TypedDict):
    messages: Annotated[list, add_messages]   # reducer: append, don't overwrite

tools = [get_order_status, refund_order]
llm = init_chat_model("anthropic:claude-sonnet-5-5").bind_tools(tools)

def agent(state: State):
    return {"messages": [llm.invoke(state["messages"])]}

builder = StateGraph(State)
builder.add_node("agent", agent)
builder.add_node("tools", ToolNode(tools))
builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", tools_condition)  # → "tools" or END
builder.add_edge("tools", "agent")

with PostgresSaver.from_conn_string(DB_URI) as checkpointer:
    checkpointer.setup()
    graph = builder.compile(checkpointer=checkpointer)
    config = {"configurable": {"thread_id": "ticket-881"}, "recursion_limit": 30}
    for update in graph.stream(inputs, config, stream_mode="updates"): ...
```

Reading the code

- `Annotated[list, add_messages]` is the reducer. Without it, every node would overwrite the conversation.
- `tools_condition` routes to `"tools"` if the last message has tool calls, otherwise to `END`.
- `ToolNode` runs the requested tools (in parallel when there are several) and appends the results.

When you don't need the graph API

For a standard loop, LangChain's prebuilt agent builds this graph for you. It replaces the older `create_react_agent`.

SHORTCUT

```python
from langchain.agents import create_agent

agent = create_agent(model="anthropic:claude-sonnet-5-5",
                     tools=[get_order_status, refund_order],
                     system_prompt="You are a support agent for Acme.")
```

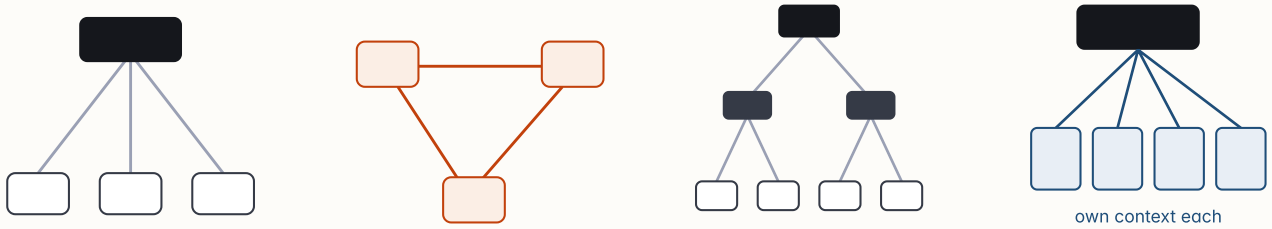
Use `StateGraph` for custom state, fixed steps or several agents.

CHECK THE DEFAULT STEP LIMIT

In `langgraph 1.2` the default `recursion_limit` is **10,007** steps (older versions used 25), so a looping agent runs until your budget does. Set it on every invoke (25–50 suits one agent) and catch `GraphRecursionError` to reply gracefully.

07 Multi-agent systems

The real benefits are context isolation and parallelism. Giving agents job titles is not the point.



Supervisor · Handoffs (swarm) · Hierarchical · Parallel sub-agents

PATTERN	HOW CONTROL MOVES	GOOD FIT
Supervisor	A central agent routes each turn to a worker; workers report back	Distinct specialist domains, such as billing vs tech support
Handoffs	The active agent transfers the conversation directly to a peer	Conversational flows where the user talks to one agent at a time
Hierarchical	Supervisors of supervisors, each team a sub-graph	Large tool surfaces split by department
Parallel sub-agents	Lead spawns workers with isolated contexts; each returns a condensed result	Breadth-first research, wide search, many independent lookups

The evidence, both ways

FOR · ANTHROPIC, 2025

An Opus 4 lead with Sonnet 4 sub-agents beat single-agent Opus 4 by **90.2%** on an internal research eval. The win came from spending more tokens in parallel, each sub-agent with a clean context window.

The same team notes it is a poor fit for tasks needing shared context or tight dependencies, "such as most coding tasks."

AGAINST · COGNITION, 2025

In *Don't Build Multi-Agents*: "actions carry implicit decisions, and conflicting decisions carry bad results." Two sub-agents building parts of one game, each without the other's context, produce pieces that don't fit.

Their advice: share full traces, not just messages, and default to a single-threaded agent.

RECONCILING THEM

Use multiple agents when subtasks are **independent and read-heavy** (search, gather, analyse). Use one agent when subtasks are **interdependent and write-heavy** (building one codebase, one document). That is the distinction that matters.

A supervisor in LangGraph

The supervisor returns a `Command`, which routes without needing conditional edges. Each worker can be a full agent graph of its own.

TEAM.PY

```python
from typing import Literal
from pydantic import BaseModel
from langgraph.types import Command

class Route(BaseModel):
    next: Literal["researcher", "writer", "FINISH"]

router = llm.with_structured_output(Route)

def supervisor(state: State) → Command[Literal["researcher", "writer", "__end__"]]:
    route = router.invoke([SUPERVISOR_PROMPT, *state["messages"]])
    return Command(goto=END if route.next == "FINISH" else route.next)

builder = StateGraph(State)
builder.add_node("supervisor", supervisor)
builder.add_node("researcher", researcher_agent)   # each can be its own graph
builder.add_node("writer", writer_agent)
builder.add_edge(START, "supervisor")
builder.add_edge("researcher", "supervisor")       # workers report back
builder.add_edge("writer", "supervisor")
team = builder.compile()
```

The delegation contract

Most multi-agent failures are bad briefs. Sub-agents start with an empty context. Whatever the lead doesn't write down, they don't know. Every delegated task should state:

```
OBJECTIVE    2025-26 e-scooter sales in India,
             by brand
OUTPUT       Table: brand | units | source URL
             (max 10 rows)
TOOLS        web_search, fetch_page only
BOUNDARIES   Sales only; pricing is another
             agent's job
STOP WHEN    Top 5 brands found, or 12 calls
```

Fields adapted from Anthropic's guidance: objective, output format, tool guidance, task boundaries.

Before adding an agent, check

- Can I name the context this agent keeps out of the main one?
- Can its work run in parallel with the others?
- Is its output easy to merge (data, not half a design)?
- Is the task valuable enough for ~15× the tokens?
- Will traces show which agent produced each fact?

If most answers are no, a single agent with better tools is usually cheaper and more reliable.

AGENTS ACROSS ORGANISATIONS

When agents are built by different teams or vendors, **A2A** (Agent2Agent, now under the Linux Foundation) standardises how one agent discovers another and hands it a task. MCP connects agents to tools. A2A connects agents to agents.

08 Shipping to production

A demo works once. Production needs it to work the ten-thousandth time, on inputs nobody tried, without surprising anyone.

Evaluate

- **Start with ~20 real queries.** Anthropic began there. Early changes have big effects that a small set catches.
- **Grade the end state** (was the refund issued correctly?) and, separately, **the trajectory** (did it call the right tools?).
- An LLM judge with a written rubric scales the work. Spot-check it against human grades.
- Turn every production incident into a new test case.

Observe

- Trace every model call, tool call and state transition, with tokens and latency (LangSmith, Langfuse, OpenTelemetry).
- Alert on step count, cost per run and tool error rate, not just uptime.

Constrain

- Explicit `recursion_limit`, plus token and wall-clock budgets per run
- Timeouts and retries with backoff on every tool
- Least privilege: scoped, read-only credentials unless writing is the job
- Approval gates on irreversible actions (Chapter 5)
- Durable checkpointer (Postgres), never in-memory

Secure

Prompt injection arrives through tool output: a web page, email or ticket that says “ignore previous instructions”. It can't be fully filtered out. Watch for the **lethal trifecta** (Simon Willison): *private data* + *untrusted content* + *a way to send data out*. An agent with all three can be tricked into leaking. Remove at least one.

Failure modes you will meet

SYMPTOM	USUAL CAUSE	FIX
Calls the same tool again and again	Unhelpful tool result; no stop condition	Return actionable errors; cap steps; detect repeated identical calls
Picks the wrong tool	Overlapping names or descriptions	Merge or rename tools; add “use when…” to descriptions
Forgets early facts on long runs	Context filled with stale tool output	Summarise, offload big results, keep the plan in state
Duplicate email or charge after resume	Side effect placed before `interrupt()`	Move it after, or use idempotency keys
Sub-agents repeat each other’s work	Vague delegation	Use the delegation contract (page 13)
Cost spikes on simple questions	No effort scaling	Effort rules in the prompt; route simple queries to a workflow
Confident but wrong answer	No grounding check	Require citations from tool output; add a verifier step

Reference architecture

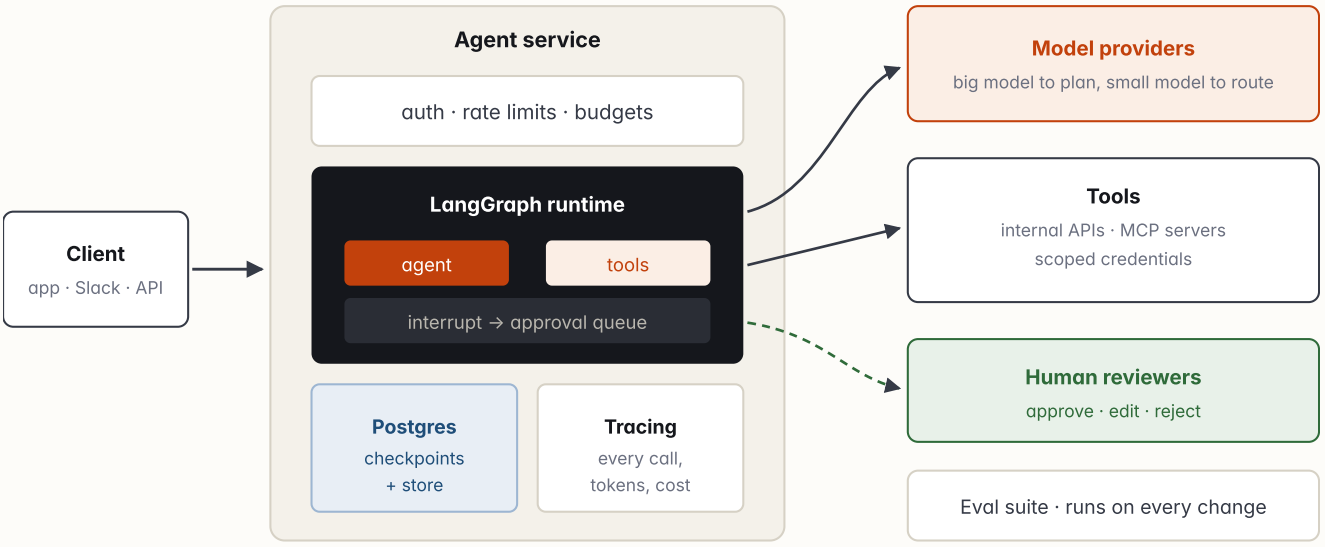
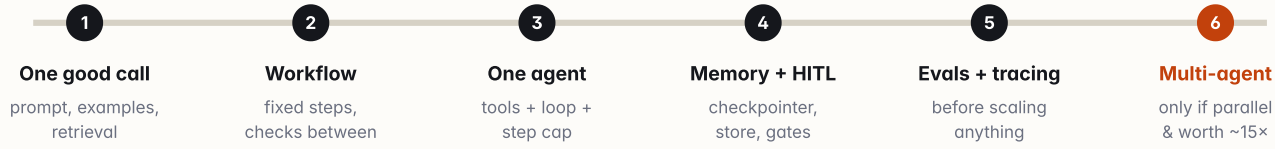


FIG 8.1 A minimal production shape. Most pieces are ordinary backend engineering; the agent is one box.

The path from zero to production



Further reading

- **Building Effective Agents**, Anthropic (2024)
- **How we built our multi-agent research system**, Anthropic (2025)
- **Don't Build Multi-Agents**, Cognition (2025)
- **LangGraph docs**: persistence, interrupts, memory, at docs.langchain.com
- **ReAct**, Yao et al. (2022) · **Reflexion**, Shinn et al. (2023)
- **CoALA**: Cognitive Architectures for Language Agents, Sumers et al. (2023)
- **The lethal trifecta**, Simon Willison (2025)

IF YOU REMEMBER FIVE THINGS

1. Use the least autonomy that solves the task.
2. Tool descriptions are prompts. Write them like it.
3. Memory is context management. Decide what goes back in.
4. Gate irreversible actions, and keep side effects after the pause.
5. No evals, no production. Start with twenty real cases.